

Pilgrimage_2

Implementation Stories and Build Guide

Design philosophy. This rebuild keeps the useful ideas from the older Pilgrimage project while avoiding the old coupling: one owner per responsibility, card data kept separate from visuals, signal-based system boundaries, explicit card states, and no hand system unless the game later proves it needs one.

Recommended Build Order

- 1 Stabilize the current card and slot foundation.
- 2 Add board/grid helpers and a board controller.
- 3 Add deck and journey-deck refill.
- 4 Add actions before effects.
- 5 Add game setup, movement, combat, and game-over rules.
- 6 Build the real game scene and keep the card test scene as a lab.

Story Details

Story 1: Stabilize Card Placement

What this achieves

Drag/drop becomes a dependable foundation: a card can enter a valid slot, invalid drops fail safely, and slot type filters use the rebuilt card data model.

Why this fits the rebuild

This protects the rebuild from one of the old project's main problems: several systems disagreeing about what a card is and where it lives. Placement should be boring, explicit, and owned by a small path.

Implementation guide

- Keep InputManager as the only drag/drop owner: `src/singletons/input_manager.gd`.
- Keep CardSlot as the only slot acceptance and placement owner: `src/board/card_slots/card_slot.gd`.
- Use `card.data` consistently; do not reintroduce old `card.cardData` naming.
- Make slot placement call `card.placeInSlot()`, reset rotation/scale, and emit `slotFilled`.
- Keep card state transitions going through `CardStateMachine` rather than direct state mutation.

Done when

- Dropping onto an empty valid slot places the card.
- Dropping onto an occupied slot leaves the card in a sane `ON_BOARD` state.
- Allowed type filters work with `card.data.type`.

- No duplicate board input controller handles drag/drop.

Story 2: Clean Board Grid API

What this achieves

Other systems can ask the board clear questions: where is the center, which slots are empty, which slots are occupied, and what neighbours exist.

Why this fits the rebuild

The rebuild should avoid ad hoc node searching. A clean grid API keeps movement, deck refill, combat, and effects from each inventing their own board scan.

Implementation guide

- Add methods to `src/board/slot_grid/slot_grid.gd`.
- Implement `get_slot_at(coordinates: Vector2i) -> CardSlot`.
- Implement `get_empty_slots()` and `get_occupied_slots()`.
- Implement `get_center_slot()` for the player start.
- Implement `get_cardinal_neighbours(slot)` for movement rules.
- Keep coordinates as `Vector2i` everywhere in the new code.

Done when

- The grid can return any slot by coordinate.
- The grid reports empty/occupied slots without outside scripts crawling children.
- The center slot is correct for the current 3x3 board.
- Movement code can use cardinal neighbours without duplicate maths.

Story 3: Add Board Controller

What this achieves

Board-level operations sit behind one small controller instead of being scattered across input, deck, and game scripts.

Why this fits the rebuild

Input should not become the game rules layer. The board controller gives the rebuild a clean boundary: Input asks, Board places, Game decides why.

Implementation guide

- Create `src/board/board_controller.gd` and a matching scene only if needed.
- Give it a `SlotGrid` reference.
- Add `place_card_in_slot(card, slot)`, `clear_board()`, and `emit_board_changed()`.
- Keep combat, effects, deck drawing, and player turn logic out of this controller.
- Use this controller from future action processing and game setup code.

Done when

- Board placement can be called without knowing slot internals.
- Clearing the board clears every occupied slot.
- Board change signals are emitted from one place.
- No combat or effect logic appears in `board_controller.gd`.

Story 4: Add Deck Model

What this achieves

The game can hold a draw pile of card ids, shuffle it, draw from it, and know when it is empty.

Why this fits the rebuild

Decks should store ids, not prebuilt card nodes. This keeps data lightweight, makes shuffling simple, and lets `CreateCard` remain the single card factory.

Implementation guide

- Create `src/decks/deck_model.gd`.
- Store an `Array[String]` of card ids.
- Implement `initialise_deck(card_ids)`, `shuffle_deck()`, `draw_card_id()`, `get_deck_size()`.
- Let `CreateCard` build the card only when a draw is resolved.
- Emit `deckShuffled/deckEmptied` through `GlobalSignalBus` when useful.

Done when

- A deck can be initialized from a known id list.
- Draw removes one id and returns it.
- Drawing an empty deck returns safely.
- Deck state is visible enough for tests/debug UI.

Story 5: Add Journey Deck

What this achieves

The board can be populated from an adventure/journey deck, replacing empty board spaces with new cards.

Why this fits the rebuild

This is the core loop's engine. It replaces hand-style card play with a board-adventure flow: reveal, move, resolve, refill.

Implementation guide

- Create `src/decks/journey_deck.gd` extending or wrapping `DeckModel`.
- Add a journey card preset list or load one from data later.
- Implement `reveal_top_card(slot)` by drawing an id, creating a card, and placing/revealing it.
- Implement `fill_empty_slots(slot_grid)` using `slot_grid.get_empty_slots()`.

- For now, direct placement is acceptable; later route through ActionQueue REVEAL_CARD.

Done when

- Starting the game can fill all empty non-player slots.
- A single empty slot can be refilled after movement.
- The deck stops cleanly when empty.
- Revealed cards enter IN_SLOT state.

Story 6: Add Action Types

What this achieves

Gameplay changes become named actions instead of loose method calls with hidden side effects.

Why this fits the rebuild

The old project pointed toward this idea but got muddy. In the rebuild, action types should become the contract between game rules, effects, animation, and board changes.

Implementation guide

- Create src/actions/action_types.gd.
- Define REVEAL_CARD, MOVE_CARD, MODIFY_STATS, DESTROY_CARD, DEAL_DAMAGE, DRAW_CARD.
- Add a helper make(type, source=null, target=null, data={}).
- Add is_valid(action) and simple warnings for unknown types.
- Keep this file data-like; do not process actions here.

Done when

- All planned action names live in one file.
- Actions have a shared shape: type, source, target, data.
- Invalid actions can be detected before processing.
- Future effects can inspect actions without knowing every caller.

Story 7: Add Action Queue

What this achieves

Gameplay events can be resolved in a stable order, which is essential once effects start reacting to other effects.

Why this fits the rebuild

A queue stops the rebuild from falling back into direct chains of script calls. It gives you one timeline for board changes, stat changes, destruction, and triggers.

Implementation guide

- Create src/singleton/actions/action_queue.gd and autoload it.
- Implement enqueue_action(action), pop_next_action(), queue_has_actions(), clear_queue().

- Reject empty or invalid actions with warnings.
- Emit actionEnqueued/actionPopped/queueCleared for debugging.
- Keep it dumb: the queue stores order, it does not decide outcomes.

Done when

- Actions are processed in first-in, first-out order.
- Bad actions do not crash the game.
- Debug output can show what action is next.
- No gameplay logic lives in the queue itself.

Story 8: Add Action Processor

What this achieves

Actions gain a single resolver that changes the game state consistently.

Why this fits the rebuild

This is the rebuild's best defense against hidden coupling. Instead of deck, movement, combat, and effects all mutating cards directly, they request actions and the processor applies them.

Implementation guide

- Create src/singletons/actions/action_processor.gd and autoload it.
- In _process, pop one action when not busy.
- Resolve REVEAL_CARD, MOVE_CARD, MODIFY_STATS, DESTROY_CARD, DEAL_DAMAGE.
- Call EffectMediator pre/post hooks later, once effects exist.
- Keep animation simple at first; correctness comes before flourish.

Done when

- REVEAL_CARD places a created card into a slot.
- MOVE_CARD moves between slots and clears the old slot.
- MODIFY_STATS refreshes card visuals after changing stats.
- DESTROY_CARD clears the slot and frees/removes the card.
- Unknown actions warn and do nothing.

Story 9: Add Effect Data Loading

What this achieves

Effect definitions in JSON become real typed data the game can use.

Why this fits the rebuild

The rebuild already has card data loading. Effects should follow the same data-driven pattern instead of becoming hard-coded exceptions on cards.

Implementation guide

- Create src/effects/effect_data.gd.
- Create src/effects/effect_data_factory.gd.
- Create src/singletons/registries/effect_data_registry.gd and autoload it.
- Load data/effect_dictionary.json.
- Resolve card.data.effects ids into EffectData entries when cards are created or placed.

Done when

- All effect JSON entries can be loaded by id.
- Missing effect ids warn clearly.
- EffectData exposes trigger, effect_type, and parameters.
- CardData stays clean and only stores effect ids.

Story 10: Add Effect Runtime

What this achieves

Cards can register effects that listen to gameplay actions and request follow-up actions.

Why this fits the rebuild

Effects are where direct coupling can get ugly fast. A mediator keeps cards from spying on every system and keeps effects reacting to the action timeline.

Implementation guide

- Create src/effects/card_effect_factory.gd.
- Create src/singletons/effects/effect_mediator.gd and autoload it.
- When a card enters the board, instantiate its effects and register listeners.
- When a card leaves/dies, unregister its listeners.
- Effects should enqueue actions, not directly edit other cards.

Done when

- A card with effect ids registers runtime effect instances.
- EffectMediator can dispatch pre/post action hooks.
- Freed cards do not leave dead listeners behind.
- Effects use ActionQueue for changes.

Story 11: Rebuild First Effects

What this achieves

The first real card effects prove the data/action/effect architecture works.

Why this fits the rebuild

Do not rebuild every effect at once. Three small effects exercise the important cases: board-count recalculation, stat modification, and destroy triggers.

Implementation guide

- Create concrete effects under `src/effects/card_effects/`.
- Implement `SolitaryBeast`: recalculates stats based on matching board cards.
- Implement `HealOnKill`: heals attacker when this host card is destroyed.
- Implement `BuffAttackOnKill`: buffs attacker when this host card is destroyed.
- Use `ActionTypes.MODIFY_STATS` for stat changes.

Done when

- Effects trigger from `ActionProcessor` pre/post hooks.
- `SolitaryBeast` updates when the board changes.
- On-kill effects modify the correct attacker.
- Destroying a card cleans up its listener.

Story 12: Add Game Controller Setup

What this achieves

A run can start from a known state: player in the center, journey cards around them, and game state set.

Why this fits the rebuild

Setup should not live in the test scene. A game controller gives the rebuild a real entry point without dragging UI or hand logic back in.

Implementation guide

- Create `src/singletons/managers/game_controller.gd` or a scene-owned controller.
- Reference `BoardController`, `SlotGrid`, and `JourneyDeck`.
- Create the player card using `CreateCard`.
- Place player in `slot_grid.get_center_slot()`.
- Fill remaining empty slots from `JourneyDeck`.
- Track states: `SETUP`, `PLAYER_TURN`, `MOVING`, `COMBAT`, `GAME_OVER`.

Done when

- Launching the main game scene sets up the board automatically.
- Player starts in the center slot.
- Other slots fill from the journey deck.
- Input is only accepted after setup completes.

Story 13: Add Movement Rules

What this achieves

The player character can move through the board one cardinal step at a time.

Why this fits the rebuild

Movement is the game's readable tactical language. Keeping it grid-based and explicit fits the no-hand, board-adventure direction.

Implementation guide

- Use `SlotGrid.get_cardinal_neighbours(player_slot)`.
- On card/slot click, determine the target slot.
- Reject diagonal and non-adjacent targets.
- For empty target slots, enqueue `MOVE_CARD`.
- After movement, enqueue a reveal/refill action for the previous slot if needed.

Done when

- Player can move left/right/up/down by one slot.
- Player cannot move diagonally.
- Player cannot jump across the board.
- The slot left behind can be refilled.

Story 14: Add Combat Rules

What this achieves

Moving into an enemy card resolves a fight and produces clear consequences.

Why this fits the rebuild

Combat should be game logic, not card UI logic. It belongs near the game controller/action layer so effects and signals can respond consistently.

Implementation guide

- Create a small combat resolver script or keep it inside `GameController` at first.
- Compare `attacker.attack` and `defender.health`.
- Apply `defender.attack` damage back to the attacker when appropriate.
- Use `DEAL_DAMAGE` and `DESTROY_CARD` actions rather than direct frees.
- Emit `battleCompleted` with result data for UI/debug.

Done when

- Attacking an occupied adjacent slot resolves combat.
- Damage changes health and refreshes visuals.
- Defeated cards are destroyed through actions.
- Player death can be detected by the game controller.

Story 15: Add Game Over Conditions

What this achieves

The game can end cleanly instead of drifting into broken states.

Why this fits the rebuild

A simple end condition gives every system a boundary. Input, deck refill, combat, and UI can all ask whether the game is over.

Implementation guide

- Add GAME_OVER to GameController state.
- When player health reaches zero, set GAME_OVER.
- Decide what empty journey deck means: win, loss, or exhausted-board state.
- Lock InputManager when game over is reached.
- Emit a gameOver signal with reason data.

Done when

- Player death ends the game.
- Input no longer moves/places cards after game over.
- Empty-deck behavior is explicit.
- The UI can display why the game ended.

Story 16: Build Main Game Scene

What this achieves

The project has a real playable scene separate from the card laboratory.

Why this fits the rebuild

The card test scene is valuable, but it should not become the architecture. A main scene lets tests stay messy and the game stay clean.

Implementation guide

- Create src/main/game_scene.tscn or update main.tscn.
- Instance SlotGrid/BoardController, JourneyDeck, and GameController.
- Wire references in the editor or via exported NodePaths.
- Keep debug buttons out of the main scene.
- Set project.godot main_scene only when the scene can initialize reliably.

Done when

- Running the main scene produces a playable board.
- The card test scene still exists for experiments.
- Game setup does not depend on test-only nodes.
- Scene ownership is clear.

Story 17: Add Minimal Game UI

What this achieves

The player can understand the current run without debug console output.

Why this fits the rebuild

Minimal UI supports the board-first philosophy without becoming a hand system or heavy menu layer. It should explain state, not take over the game.

Implementation guide

- Add a small UI layer to the main game scene.
- Show journey deck count.
- Show current game state or prompt.
- Show recent combat/result text.
- Show game over reason.
- Avoid complex menus until the core loop is fun.

Done when

- The player can see deck count.
- The player can tell when input is expected.
- Combat feedback appears on screen.
- Game over is visible without checking logs.

Story 18: Keep Card Test Scene As A Lab

What this achieves

You keep a safe place to test cards, visuals, drag/drop, and effects without touching the real game scene.

Why this fits the rebuild

This is a pressure valve for experimentation. The rebuild stays clean when experiments live in a known test scene instead of leaking into production systems.

Implementation guide

- Keep `src/tests/card_test_scene.tscn` and `card_test_scene.gd`.
- Use it for card creation, hover, drag/drop, stat changes, and later effect checks.
- Mark debug controls clearly and keep them under `src/tests`.
- Do not make main gameplay depend on `card_test_scene.gd`.
- Remove noisy prints once a behavior is no longer under active investigation.

Done when

- The test scene can still spawn sample cards.

- The main scene does not require test nodes.
- Visual and effect experiments have a place to live.
- Debug helpers are easy to delete later.

Guardrails

Use this checklist whenever a story touches shared architecture. It is deliberately small: the goal is to keep the rebuild clear while still moving quickly.

- One owner per responsibility: Input owns pointer drag, Board owns placement, Game owns rules, Actions own state mutation flow.
- Prefer data ids and factories over prebuilt scene references for cards and effects.
- Do not bring back the hand system as a hidden dependency.
- Do not let the card test scene become the main architecture.
- Effects should enqueue actions rather than directly editing unrelated cards.
- Keep debug helpers in src/tests or remove them once a behavior is stable.